# Practical - 12

## Event Management CRUD with Poster Upload and File Validation

**Problem Definition :** Develop event management functionality for admin users. Admin can add, view, update, delete, and upload event posters with file type and file size validation.

## A. Theory

### 1. Are add, update, and delete functionalities implemented correctly?

Yes — all 3 are implemented:

- Add → add_event.php runs INSERT INTO events
- Update → edit_event.php pre-fills from SELECT, then runs UPDATE events
- Delete → events.php handles ?delete=id, runs DELETE FROM events, and also deletes the poster file from the UPLOADS folder using unlink()

### 2. Is the UI linked with the database correctly?

Yes — events.php fetches all events using SELECT * FROM events and displays them in an HTML table with poster thumbnails, title, category, date, and action buttons (Edit/Delete) — all pulled directly from the MySQL database.

### 3. Are poster uploads validated for type and size?

Yes — before moving the uploaded file, the script checks:

- File type: only jpg, jpeg, png, gif are accepted using pathinfo() and in_array()
- File size: must be under 2MB (2 × 1024 × 1024 bytes)
- If either check fails, an error message is displayed and no file is saved

### 4. Are success and failure messages displayed?

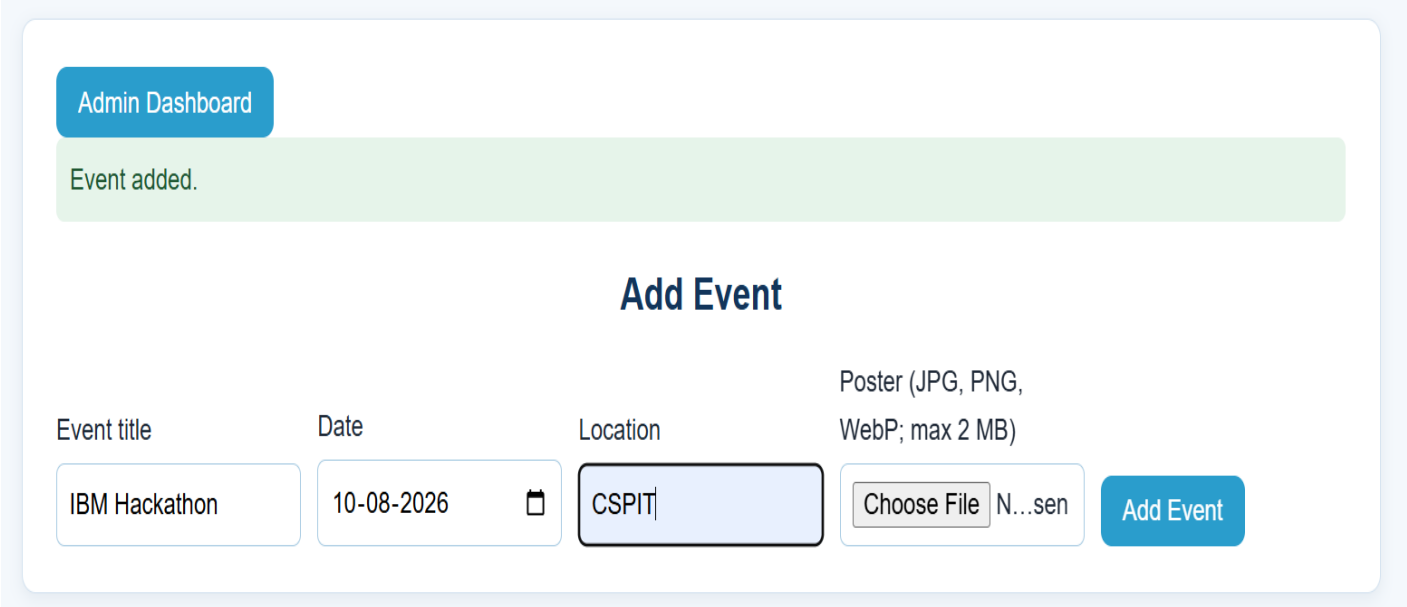
Yes — all three pages display color-coded messages: green for success, red for errors. Error cases include wrong file type, file too large, empty required fields, and database errors.

## B. Implementation

| | |
|---|---|
| db_connect.php | Reused — basic MySQLi |

| | |
|---|---|
| events.php | Lists all events, handles DELETE |
| add_event.php | Form to INSERT event + upload |
| edit_event.php | Pre-filled form for UPDATE event + |
| update_events_table.sql | Adds poster column to existing |
| UPLOADS/ | Folder where uploaded poster |

## C. Evidence of Work (Screenshots)






## D. GitHub Repository Link:

https://github.com/25dce037-tech/Student-Hub-Portal.git

## E. Challenges Faced:

1. File Upload with enctype — The HTML form must use enctype="multipart/form-data" for file uploads to work. Without this, $_FILES is always empty — a very common beginner mistake.
2. Unique Filenames — Using time() prefix (1728835200_poster.jpg) prevents two uploads from overwriting each other if they have the same filename.
3. Deleting Old Poster on Update — When a new poster is uploaded during edit, the old file must be deleted from disk using unlink(). Forgetting this would leave orphan files in the UPLOADS folder.
4. UPLOADS folder permissions — The folder must have chmod 777 so that PHP (running as the Apache user) is allowed to write files into it.

## F. Learning Outcomes:

1. Learned complete CRUD for a different entity (Events instead of Students) — reinforcing the pattern.
2. Learned file uploads in PHP using $_FILES, move_uploaded_file(), and enctype="multipart/formdata".
3. Understood file validation — checking both file extension (pathinfo) and file size ($_FILES['poster']['size']).

4. Learned how to manage uploaded files — creating unique names with time(), storing only the filename in the database, and deleting files with unlink().
5. Learned to display thumbnail previews of uploaded images in the HTML table.

## Faculty Remarks: